

MiniVue — Vue 3 своими руками

Строим фреймворк с нуля: реактивность, virtual DOM, компилятор, компоненты, роутер, стор и SSR

Серик Мурадов

Содержание

Зачем строить Vue заново	1
Что мы построим	1
Правила игры: без сборки	2
Как устроена книга и репозиторий	2
Для кого это	2
Минимум JavaScript	3
Значения и переменные	3
Функции	3
Объекты	4
Массивы	4
Деструктуризация и spread	5
Классы, геттеры и сеттеры	5
Set, Map и WeakMap	6
Symbol	6
Proxy и Reflect	7
Object.is	7
Модули: import и export	8
optional chaining и !!	8
Реактивность	9
Задача в одном предложении	9
Два глагола: track и trigger	9
Эффект — это функция, которую можно перезапустить	10
Где живут связи: targetMap	11
reactive: перехват объекта через Proxy	13
ref: реактивность для одного значения	14
computed: лениво и с кэшем	15
Почему нужен cleanup: задача с ветвлением	16
watch: следить и получать старое/новое	16
Как всё это работает вместе	17
Проверяем себя	17

Зачем строить Vue заново

Есть два способа понять инструмент. Первый — прочитать инструкцию и начать им пользоваться. Второй — разобрать его на части и собрать самому. Эта книга про второй способ. Мы не будем учиться писать приложения на Vue — мы напишем сам Vue.

Звучит страшно, но пугаться нечего. Vue внутри — это не магия, а несколько простых идей, аккуратно сложенных друг на друга. Реактивность («данные изменились — экран обновился сам») — это перехват чтения и записи. Компоненты — это функции, которые возвращают описание разметки. Роутер — это словарь «адрес → что показать». Каждую из этих идей можно уместить в сотню-другую строк, которые читаются как обычный текст. Мы пройдем их все.

Что мы построим

К концу книги у нас будет работающий мини-фреймворк со всей экосистемой настоящего Vue 3:

- **Реактивность** — `ref`, `reactive`, `computed`, `watch`. Ядро, из-за которого Vue вообще существует.
- **Virtual DOM и рендерер** — как описание интерфейса превращается в реальные элементы страницы, и как обновлять их точно, а не перерисовывать всё.
- **Компоненты** — `setup`, входные параметры (`props`), события (`emit`), слоты, хуки жизненного цикла, `createApp`.
- **Компилятор шаблонов** — как `<div>{{ count }}</div>` превращается в функцию.
- **Роутер** — аналог Vue Router: адреса, параметры, `RouterView`, `RouterLink`.
- **Стор** — аналог Pinia: общее состояние приложения.
- **SSR** — рендеринг на сервере и «оживление» (гидратация) на клиенте.

Имена мы намеренно берём такие же, как в настоящем Vue. Наш `ref` называется `ref`, наш `createApp` называется `createApp`. Поэтому всё, что вы поймёте здесь, один в один переносится на реальный фреймворк — только там кода больше, потому что он обрабатывает пограничные случаи, которые мы иногда будем опускать ради ясности.

Правила игры: без сборки

Обычно, чтобы запустить Vue-проект, ставят Node, потом сборщик (Vite или Webpack), и он «магически» превращает ваши файлы в то, что понимает браузер. Для новичка это худшее начало: половина происходящего спрятана внутри инструмента, который вы ещё не понимаете.

Поэтому у нас правило: **никакой сборки**. Мы пишем на чистых ES-модулях — стандарте, который браузер понимает сам. Файл с кодом подключается к странице строчкой `<script type="module">`, а внутри работает `import`. Открыли страницу — код поехал. Всё, что происходит, происходит у вас на глазах, без посредников.

Node.js нам понадобится ровно для двух вещей: запускать тесты и поднимать крошечный локальный сервер, чтобы браузер разрешил загрузку модулей (со схемы `file://` он их из соображений безопасности не грузит). Сам фреймворк от Node не зависит — до главы про SSR, где сервер и есть суть.

Как устроена книга и репозиторий

Каждый слой — это одновременно:

- **глава** здесь, в книге, где идея объясняется словами и картинками;
- **папка в `packages/`** с рабочим кодом, обильно закомментированным на русском;
- **тесты в `test/`**, которые доказывают, что код делает то, что обещано;
- **демо в `playground/`**, которое можно открыть в браузере и потрогать руками.

Советую держать рядом код и главу: прочитали объяснение — открыли файл, нашли те самые строки. Код в книге и код в репозитории — это буквально одни и те же строки.

Для кого это

Для того, кто хочет понимать, а не только применять. Если вы никогда не писали на JavaScript — не страшно: следующая глава даёт ровно тот минимум языка, который нужен, чтобы читать наш код. Если вы уже пишете на Vue, но он для вас «чёрный ящик» — эта книга ящик открывает.

Одно предупреждение: не пытайтесь запомнить всё сразу. Реактивность в первой главе поначалу кажется головоломкой — это нормально. Дойдите до демо, потыкайте кнопки, вернитесь к коду. Идея встанет на место. А когда встанет — всё остальное во Vue станет очевидным следствием.

Поехали.

Минимум JavaScript

Эта глава — не полноценный курс языка, а словарь. Здесь ровно те конструкции, которые встретятся в нашем коде, и ни одной лишней. Если вы уже знаете JavaScript, пролистайте её и переходите к реактивности. Если нет — прочитайте один раз, а дальше возвращайтесь как к справочнику, когда в коде встретится незнакомый значок.

Значения и переменные

Значение — это кусочек данных: число 42, строка "привет", логическое true или false, «ничего» — null и undefined. Переменная — это имя для значения.

```
const name = 'Аня'    // const — имя, которое нельзя переназначить
let count = 0         // let — имя, которое можно менять
count = count + 1     // теперь count равно 1
```

const не значит «значение неизменно» — он значит «нельзя присвоить это имя заново». Объект под именем const менять можно; нельзя лишь сказать имя = другой объект. По умолчанию берите const, а let — только когда правда нужно переприсваивать.

Функции

Функция — это заготовка действий, которую можно запускать много раз, подставляя разные входные значения (аргументы).

```
function double(x) {
  return x * 2
}
double(21) // 42
```

Есть короткая запись — стрелочная функция. Она нам будет попадаться постоянно.

```
const double = (x) => x * 2      // одно выражение – return подразумевается
const log = (msg) => {           // тело в фигурных скобках – return пишем сами
  console.log(msg)
}
const now = () => Date.now()     // без аргументов – пустые скобки
```

Функцию можно передать другой функции как аргумент — это и есть «колбэк», функция-обработчик. Вся реактивность построена на том, что мы принимаем чужую функцию и решаем, когда её вызвать:

```
effect(() => {
  console.log('перезапусти меня, когда данные изменятся')
})
```

Объекты

Объект — набор пар «ключ: значение». Ключи (свойства) читают и пишут через точку.

```
const user = { name: 'Аня', age: 30 }
user.name    // 'Аня' – чтение
user.age = 31 // запись
user.city = 'Алматы' // добавление нового свойства
```

Запомните слова **чтение свойства** и **запись свойства** — вокруг них крутится вся первая глава. Реактивность — это перехват именно этих двух операций.

Массивы

Массив — упорядоченный список значений. Индексация с нуля.

```
const items = [10, 20, 30]
items[0]      // 10
items.length  // 3
items.push(40) // добавить в конец → [10, 20, 30, 40]
items.reduce((sum, x) => sum + x, 0) // 100 – «свернуть» в одно число
```

reduce идёт по элементам и накапливает результат: начинаем с 0, к каждому элементу прибавляем его значение. Так мы будем считать сумму корзины.

Деструктуризация и spread

Можно «разобрать» объект или массив на отдельные переменные:

```
const { name, age } = user      // name = 'Аня', age = 31
const [first, second] = items  // first = 10, second = 20
```

Три точки ... — «разложи остальное». В нашем коде она встречается как «сделать копию»:

```
const copy = [...items]        // новый массив с теми же элементами
```

Эта строчка спасёт нас в реактивности: перед перебором набора эффектов мы копируем его, чтобы изменения во время перебора не сломали цикл.

Классы, геттеры и сеттеры

Класс — шаблон для создания объектов с общим поведением. `new` создаёт экземпляр, `constructor` задаёт начальное состояние, `this` внутри — «этот конкретный экземпляр».

```
class Counter {
  constructor(start) {
    this.value = start  // поле экземпляра
  }
  increment() {
    this.value++
  }
}
const c = new Counter(5)
c.increment()
c.value // 6
```

Особая пара — **геттер** и **сеттер**. Это методы, которые снаружи выглядят как обычное свойство, но при чтении/записи выполняют код. Именно так устроен `ref`: обращение к `.value` выглядит как поле, а на деле запускает функцию.

```
class Box {
  constructor(v) { this._v = v }
  get value() {      // сработает при ЧТЕНИИ box.value
    console.log('прочитали')
    return this._v
  }
  set value(newV) {   // сработает при ЗАПИСИ box.value = ...
  }
}
```

```
    console.log('записали')
    this._v = newV
  }
}
const box = new Box(1)
box.value    // печатает «прочитали», возвращает 1
box.value = 2 // печатает «записали»
```

Подчёркивание в `_v` — просто договорённость «это внутреннее поле, снаружи не трогай». Язык его не защищает, но так принято.

Set, Map и WeakMap

Кроме массивов и объектов нам нужны три специальные коллекции.

Set — набор уникальных значений. Добавили дважды — хранится один раз. Идеально для «списка эффектов, подписанных на свойство»: один эффект не должен попасть туда дважды.

```
const s = new Set()
s.add('a'); s.add('a'); s.add('b')
s.size    // 2
s.has('a') // true
s.delete('b')
```

Map — как объект, но ключом может быть что угодно, включая другой объект. Нам это критично: мы будем связывать реактивный объект с его зависимостями.

```
const m = new Map()
m.set('ключ', 123)
m.get('ключ') // 123
```

WeakMap — то же, что Map, но со «слабыми» ключами-объектами: если на объект-ключ больше никто не ссылается, сборщик мусора вправе удалить его вместе с записью. Так хранилище зависимостей не мешает освободить память от объектов, которые больше не нужны.

Symbol

`Symbol('имя')` создаёт уникальное значение-метку, которое гарантированно не совпадёт ни с каким строковым ключом. Мы используем символы как служебные «скрытые» свойства —

например, метку «этот объект уже реактивный», которую не спутать с настоящими данными пользователя.

```
const IS_REACTIVE = Symbol('isReactive')
obj[IS_REACTIVE] // никакое обычное свойство сюда не попадёт случайно
```

Proxy и Reflect

Вот главный инструмент реактивности. **Proxy** оборачивает объект и позволяет перехватить операции над ним — в первую очередь чтение (get) и запись (set).

```
const raw = { count: 0 }
const proxy = new Proxy(raw, {
  get(target, key) {
    console.log('читают', key)
    return target[key]
  },
  set(target, key, value) {
    console.log('пишут', key, '=', value)
    target[key] = value
    return true // set обязан вернуть true при успехе
  },
})
proxy.count // печатает «читают count», возвращает 0
proxy.count = 5 // печатает «пишут count = 5»
```

Из этого крошечного перехвата и вырастает reactive: в get мы будем вызывать «запомни, кто это прочитал», а в set — «разбуди тех, кто это читал».

Reflect — набор функций, повторяющих стандартные операции. `Reflect.get(target, key, receiver)` — то же, что `target[key]`, но корректно передаёт `this` в геттеры и работает с наследованием. Внутри Proxy правильно использовать именно Reflect, а не `target[key]` напрямую — иначе на хитрых объектах будут ошибки.

Object.is

Сравнение «изменилось ли значение». Обычный `===` почти везде подходит, но у него есть исторические странности: `NaN === NaN` даёт `false`, хотя это «то же самое». `Object.is` их чинит. Мы будем спрашивать «значение действительно другое?» через `!Object.is(старое, новое)`, чтобы не будить интерфейс на присваивании того же самого.

Модули: `import` и `export`

Код разбит на файлы-модули. Файл делится тем, что он `export`, а забирает чужое через `import`.

```
// effect.js
export function track() { /* ... */ }
export let activeEffect = undefined

// reactive.js
import { track, activeEffect } from './effect.js'
```

Путь пишется с расширением `.js` и с `./` для соседних файлов — так требует браузерный стандарт ES-модулей, которым мы пользуемся без сборщика.

`optional chaining` и `!!`

`a?.b` читает `a.b`, но если `a` — `null` или `undefined`, спокойно возвращает `undefined` вместо ошибки. `!!x` превращает любое значение в честный `true/false` (два «не»: первое инвертирует, второе возвращает обратно). Встретите `return !! (value && value.__isRef)` — это просто «верни `true`, если у `value` есть флаг `__isRef`».

Этого словаря хватит, чтобы прочитать любую строчку нашего фреймворка. Не забывайте всё — держите главу под рукой и возвращайтесь по мере надобности. А теперь — к сердцу Vue.

Реактивность

Всё во Vue держится на одной способности: данные изменились — то, что от них зависит, обновилось само. Вы меняете `count`, и число на экране становится другим, хотя вы нигде не писали «найди элемент и обнови текст». Эта глава — про то, как это устроено. Мы построим реактивность целиком, и дальше весь фреймворк будет лишь надстройкой над ней.

Код этой главы живёт в `packages/reactivity/`. Тесты — в `test/reactivity.test.mjs`, живое демо — `playground/01-reactivity.html`.

Задача в одном предложении

Мы хотим уметь писать так:

```
const count = ref(0)

effect(() => {
  document.title = 'Кликов: ' + count.value
})

count.value = 5  // и заголовок вкладки сам стал «Кликов: 5»
```

Мысль простая. Есть данные (`count`) и есть функция, которая их использует (`effect`). Мы хотим, чтобы при изменении данных функция запустилась заново — сама, без нашего участия. Значит, системе нужно как-то узнать, что эта функция «зависит» от `count`. А узнать это можно ровно в один момент — когда функция читает `count.value`. Вокруг этой мысли строится всё.

Два глагола: `track` и `trigger`

Запомните два слова, дальше книга крутится вокруг них.

- **track** («отследить») — происходит при **чтении** данных. Смысл: «сейчас выполняется какая-то функция, и она прочитала вот это значение — запомним связь».
- **trigger** («запустить») — происходит при **записи** данных. Смысл: «значение изменилось — найдём все функции, которые его читали, и запустим их заново».

Читаем — запоминаем связь. Пишем — используем связь. Всё остальное — детали хранения этих связей.

Эффект — это функция, которую можно перезапустить

Функцию, которую надо перезапускать при изменениях, мы называем **эффектом**. Чтобы system'a могла ей управлять, оборачиваем её в объект. Так к функции получится прицепить служебные данные.

Ключевая переменная — `activeEffect`. Пока выполняется тело эффекта, в ней лежит этот эффект. Любое чтение данных в этот момент знает, «кому» приписать зависимость. Вне эффектов там `undefined`, и чтение ничего не отслеживает.

```
// packages/reactivity/effect.js
export let activeEffect = undefined
const effectStack = []
```

Зачем стек? Эффекты бывают вложенными (компонент, внутри него — вычисляемое значение). Когда внутренний эффект отработал, «активным» снова должен стать внешний. Стек это и обеспечивает: положили при входе, сняли при выходе.

Сам эффект — класс `ReactiveEffect`:

```
export class ReactiveEffect {
  constructor(fn, scheduler = null) {
    this.fn = fn
    this.scheduler = scheduler
    this.deps = []
    this.active = true
  }

  run() {
    if (!this.active) return this.fn()
    cleanup(this) // забыть старые зависимости
    try {
      effectStack.push(this)
      activeEffect = this // «теперь слушаю я»
    }
  }
}
```

```
    return this.fn()           // выполняем — внутри произойдут track'и
  } finally {
    effectStack.pop()
    activeEffect = effectStack[effectStack.length - 1]
  }
}
```

Магия — в методе `run`. Перед вызовом функции он объявляет себя активным (`activeEffect = this`). Пока функция работает и читает данные, каждое чтение видит этот `activeEffect` и записывает связь. Как функция закончила — снимаем себя со стека и возвращаем активным того, кто был до нас. Обёртка `try/finally` гарантирует, что активный эффект восстановится, даже если внутри случится ошибка.

Про `scheduler` (планировщик) и `cleanup` поговорим чуть ниже — сначала посмотрим, как хранятся связи.

Где живут связи: `targetMap`

Нам нужно хранить, какие эффекты зависят от какого свойства какого объекта. Структура — три уровня вложенности:

```
targetMap: WeakMap {
  реактивныйОбъект → depsMap: Map {
    'свойство' → dep: Set(эффект1, эффект2, ...)
  }
}
```

Читается так: «для этого объекта, для этого его свойства — вот набор эффектов, которые его читали». Верхний уровень — `WeakMap`, чтобы забытые объекты не держались в памяти. Средний — `Map` по именам свойств. Нижний — `Set` эффектов (уникальных, без повторов).

Функция `track` наполняет эту структуру:

```
export function track(target, key) {
  if (!activeEffect) return           // никто не слушает — выходим

  let depsMap = targetMap.get(target)
  if (!depsMap) targetMap.set(target, (depsMap = new Map()))

  let dep = depsMap.get(key)
```

```
if (!dep) depsMap.set(key, (dep = new Set()))

trackEffects(dep)           // связать активный эффект с этим dep
}

export function trackEffects(dep) {
  if (!activeEffect) return
  dep.add(activeEffect)      // dep знает про эффект
  activeEffect.deps.push(dep) // эффект знает про dep
}
```

Обратите внимание на двустороннюю связь в `trackEffects`. `dep` запоминает эффект — чтобы потом его разбудить. Но и эффект запоминает `dep` (в своём массиве `deps`) — это понадобится для очистки, о которой ниже.

Обратная операция — `trigger`:

```
export function trigger(target, key) {
  const depsMap = targetMap.get(target)
  if (!depsMap) return           // объект никто не читал
  const dep = depsMap.get(key)
  if (!dep) return
  triggerEffects(dep)
}

export function triggerEffects(dep) {
  const effects = [...dep]       // копия! (объясняется ниже)
  for (const effect of effects) {
    if (effect === activeEffect) continue // не будим сами себя
    if (effect.scheduler) effect.scheduler()
    else effect.run()
  }
}
```

Две тонкости, каждая из которых иначе стоила бы часов отладки:

1. **Копия набора** (`[...dep]`) перед перебором. Когда эффект перезапускается, он в `cleanup` удаляет себя из `dep` и тут же в `track` добавляется обратно. Менять `Set`, по которому прямо сейчас идёт цикл, — верный способ получить бесконечный цикл или пропуск. Перебираем копию — оригинал пусть меняется спокойно.

2. `if (effect === activeEffect) continue` — защита от самоперезапуска. Если внутри эффекта написать `count.value++`, он одновременно и читает, и пишет `count`. Без этой строчки он будил бы сам себя до бесконечности.

reactive: перехват объекта через Proxy

Теперь заставим обычный объект вызывать `track` и `trigger` автоматически. Инструмент — Proxy. В перехватчике `get` зовём `track`, в `set` — `trigger`.

```
// packages/reactivity/reactive.js
export function reactive(target) {
  if (!isObject(target)) return target
  // ... защита от повторной обёртки ...

  return new Proxy(target, {
    get(obj, key, receiver) {
      const result = Reflect.get(obj, key, receiver)
      track(obj, key) // «меня прочитали»
      if (isObject(result)) return reactive(result) // глубина по требованию
      return result
    },
    set(obj, key, value, receiver) {
      const oldValue = obj[key]
      const result = Reflect.set(obj, key, value, receiver)
      if (hasChanged(oldValue, value)) trigger(obj, key) // «меня изменили»
      return result
    },
  })
}
```

Три решения, которые стоит проговорить.

Глубокая реактивность по требованию. Если прочитанное свойство само оказалось объектом, мы оборачиваем его в `reactive` прямо в этот момент — не заранее, рекурсивно по всему дереву (это дорого и иногда вредно), а лениво, когда до него реально дотянулись. Поэтому `state.user.name` реактивен на всех уровнях, но за это не платится цена, пока к `user` не обратились.

Проверка `hasChanged`. `trigger` зовём только если значение действительно другое. Присваивание того же самого не должно будить интерфейс.

Reflect вместо `obj[key]`. На объектах с геттерами и наследованием прямой доступ теряет правильный `this`. `Reflect.get/set` с `receiver` его сохраняют. На простых объектах разницы нет, но привыкать стоит к правильному варианту.

ref: реактивность для одного значения

Proxy перехватывает свойства объекта. А как сделать реактивным простое число? У числа нет свойств, которые можно перехватить. Решение — положить значение в объект, в поле `.value`, и следить за чтением/записью этого поля через геттер и сеттер.

```
// packages/reactivity/ref.js
class RefImpl {
  constructor(value) {
    this._value = convert(value) // объект внутри ref тоже делаем реактивным
    this._rawValue = value
    this.dep = new Set()        // свой набор эффектов (у ref одно «свойство»)
    this.__isRef = true
  }
  get value() {
    if (activeEffect) trackEffects(this.dep) // чтение → track
    return this._value
  }
  set value(newValue) {
    if (hasChanged(toRaw(newValue), this._rawValue)) {
      this._rawValue = toRaw(newValue)
      this._value = convert(newValue)
      triggerEffects(this.dep) // запись → trigger
    }
  }
}
```

Вот почему у `ref` всегда пишут `.value`: это не каприз, а единственная зацепка, через которую геттер и сеттер могут вклиниться и вызвать `track/trigger`. У `ref` свой `dep` прямо в объекте — ему не нужен `targetMap`, ведь «свойство» у него ровно одно.

Есть удобная надстройка `proxyRefs` — она разворачивает `.value` автоматически, чтобы в шаблонах писать `count`, а не `count.value`. Позже слой компонентов применит её к результату `setup`, и обращение к `.value` в разметке исчезнет. Код у неё короткий, загляните в `ref.js`.

computed: лениво и с кэшем

Вычисляемое значение — это формула поверх других реактивных данных, у которой два полезных свойства: она считается **лениво** (только когда результат спросят) и **кэшируется** (пока зависимости не изменились, повторный запрос отдаёт готовое).

Собирается оно из уже готовых деталей: эффект с ленивым запуском и планировщиком плюс флаг «грязно».

```
// packages/reactivity/computed.js
class ComputedRefImpl {
  constructor(getter) {
    this._dirty = true           // надо ли пересчитать
    this.dep = new Set()
    this.__isRef = true
    this.effect = new ReactiveEffect(getter, () => {
      if (!this._dirty) {       // зависимость изменилась →
        this._dirty = true      // помечаем «грязно»
        triggerEffects(this.dep) // и будим тех, кто читал computed
      }
    })
  }
  get value() {
    if (activeEffect) trackEffects(this.dep)
    if (this._dirty) {          // пересчитываем только если грязно
      this._value = this.effect.run()
      this._dirty = false
    }
    return this._value
  }
}
```

Здесь впервые заигрывает **планировщик** (второй аргумент `ReactiveEffect`). Обычный эффект при изменении зависимости сразу перезапускается. Но `computed` не должен считаться немедленно — он должен считаться лениво. Поэтому вместо пересчёта его планировщик лишь поднимает флаг `_dirty` и будит читателей. Сам пересчёт случится в `get value`, и только если кто-то действительно спросил результат. Прочитали дважды без изменений — второй раз вернётся из кэша, `getter` не выполнится.

Почему нужен `cleanup`: задача с ветвлением

Вернёмся к `cleanup`, который `run` зовёт перед каждым запуском. Рассмотрим эффект с условием:

```
effect(() => {  
  out.value = show.value ? a.value : b.value  
})
```

Когда `show` истинно, эффект читает `show` и `a`, но **не** `b`. Значит, подписываться на `b` он не должен — иначе изменение `b`, которое сейчас ни на что не влияет, зря его перезапустит. Но при следующем запуске `show` может стать ложным, и тогда нужен `b`, а не `a`.

Вывод: перед каждым запуском эффект должен забыть все свои прошлые зависимости и собрать их заново — только те, что реально прочитаны в этот раз. Это и делает `cleanup`:

```
function cleanup(effect) {  
  for (const dep of effect.deps) dep.delete(effect)  
  effect.deps.length = 0  
}
```

Он проходит по всем `dep`, где эффект записан (вот зачем эффект хранил обратные ссылки в `deps`), вычёркивает себя оттуда и очищает свой список. После `cleanup` эффект «ни на что не подписан» — а затем `fn()` подпишет его заново, но уже по факту чтения. В тестах это проверяет случай «ветвление — отписка от неиспользуемой зависимости»: после переключения ветки изменение старой переменной эффект больше не будит.

`watch`: следить и получать старое/новое

`effect` просто перезапускает функцию. `watch` — надстройка, которая на изменение зовёт колбэк и передаёт в него новое и старое значения. Источником может быть `ref`, геттер-функция или целый `reactive`-объект.

Идея: любой источник приводим к функции-геттеру, оборачиваем в эффект, а всю «реакцию» кладём в планировщик — он вычислит новое значение и позовёт колбэк.

```
// packages/reactivity/watch.js (сокращённо)  
let oldValue  
const job = () => {  
  const newValue = effect.run()  
  callback(newValue, oldValue)  
  oldValue = newValue  
}
```

```
}  
const effect = new ReactiveEffect(getter, job)  
oldValue = effect.run()           // первый прогон: собрать зависимости, запомнить старт
```

Для reactive-объекта, чтобы следить «глубоко», используется обход `traverse` — он рекурсивно читает все вложенные свойства, тем самым подписывая эффект на каждый уровень. `immediate: true` заставляет колбэк сработать сразу, не дожидаясь первого изменения.

Как всё это работает вместе

Соберём картину на примере из демо. Есть `count = ref(0)`, есть `computed doubled = count.value * 2`, и есть эффект, который пишет оба числа в DOM.

1. Эффект запускается первый раз. `activeEffect` — это он. Он читает `count.value` (→ `track: count.dep` запомнил эффект) и `doubled.value`.
2. Чтение `doubled.value` при `_dirty = true` запускает его внутренний эффект, который читает `count.value` (→ `count.dep` запомнил и `computed`-эффект). Результат кэшируется, `_dirty = false`.
3. Пользователь жмёт «+1»: `count.value = 1`. Сеттер `ref` зовёт `triggerEffects`.
4. В `count.dep` теперь два «слушателя»: наш DOM-эффект и `computed`-эффект. У `computed` есть планировщик — он ставит `_dirty = true` и будит читателей `doubled`. У DOM-эффекта планировщика нет — он просто перезапускается.
5. DOM-эффект читает `doubled.value`. Тот `_dirty`, пересчитывается (2), кэшируется. Числа на экране обновились.

Ни одной строчки «найди элемент, поставь текст» мы при клике не писали. Мы лишь описали зависимости, а система сама протянула изменение по цепочке. В этом вся суть Vue — и мы её только что построили.

Проверяем себя

Запустите тесты:

```
npm test
```

Тринадцать проверок в `test/reactivity.test.mjs` подтверждают каждое свойство: `ref` и `reactive` реагируют, `computed` ленив и кэширует, `cleanup` отписывается от лишнего, `watch` отдаёт старое и новое. Потом откройте демо:

```
npm run serve  
# http://localhost:5173/playground/01-reactivity.html
```

Понажимайте кнопки, посмотрите в консоль, как срабатывает эффект. Когда почувствуете, что реактивность «щёлкнула», переходите к следующему слою — virtual DOM. Там мы научимся превращать данные не в `document.title`, а в целые деревья элементов, и обновлять их точно.